LEBOHANG TANGU

Distributed Computation On Low-End Processors

Introduction

This study examines the computational performance of consumer-grade processors when executing matrix multiplications at GPT-1 scale. The goal is to determine whether low-end CPUs can meaningfully handle the linear algebra operations that underpin transformer-based models, and to quantify the performance gap between processor generations.

Methodology

Benchmarks were conducted using Python and NumPy, simulating core GPT-1 matrix operations. Three matrix shapes representing the input projection, attention, and feedforward layers were each run 100 times and averaged. Parameters such as CPU thermal state and background processes remained uncontrolled.

Hardware

The Intel Celeron N3350 is a low-power dual-core processor released in 2016, designed for entry-level computing tasks. It operates at a base frequency of 1.1 GHz with a turbo boost up to 2.4 GHz and features integrated Intel HD Graphics 500. It represents the lower bound of practical consumer hardware.

The AMD Ryzen 7 5000 series is a high-performance 8-core processor built on AMD's Zen 3 architecture. Designed for gaming, content creation, and multitasking, it offers significantly higher clock speeds, wider SIMD units, and improved cache architecture compared to the Celeron, representing a current mid-to-high range consumer CPU.

Results

Shape	Represents	N3350	Ryzen 7	Speedup
(512, 768)	Input projection	24.07ms	2.41ms	10 ×
(768, 768)	Attention weights	53.01ms	4.22ms	12.5 ×
(768, 3072)	Feedforward expansion	230.44ms	14.95ms	15 ×

Distributed Computation On Low-End Processors

Analysis and Conclusion

The Ryzen 7 outperformed the N3350 by 10–15×, with the gap widening on larger matrices. The feedforward layer was the costliest operation on both platforms. These findings suggest low-end processors alone are insufficient for LLM inference, but could potentially contribute within a distributed system where computation is partitioned across multiple nodes.

```
import numpy as np
import time
sizes = [
   (512, 768),
   (768, 768),
   (768, 3072),
   (3072, 768),
]
def benchmark(func, A, B, runs=100):
   _ = func(A, B)
   start = time.perf_counter()
   for _ in range(runs):
      func(A, B)
   end = time.perf_counter()
   return ((end - start) / runs) * 1000
print("Matrix test ")
print("=" * 60)
for shape in sizes:
   A = np.ascontiguousarray(np.random.randn(*shape).astype(np.float32))
   B = np.ascontiguousarray(np.random.randn(shape[1],
shape[0]).astype(np.float32))
   t1 = benchmark(np.dot, A, B)
   t2 = benchmark(np.matmul, A, B)
   t3 = benchmark(lambda a, b: a @ b, A, B)
   print(f"Shape {shape}:")
   print(f"  dot:    {t1:.2f}ms")
   print(f"  matmul: {t2:.2f}ms")
   print(f"  @:      {t3:.2f}ms")
print("=" * 60)
print("Done!")
```